HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

BÁO CÁO GIỮA KỲ

Xây dựng Chatbot hỏi đáp về lịch sử Việt Nam sử dụng GraphRAG

Lớp: E22HTTT

Tên: Nguyễn Đức Anh

ID: B22DCDT017

Người hướng dẫn: Kim Ngọc Bách

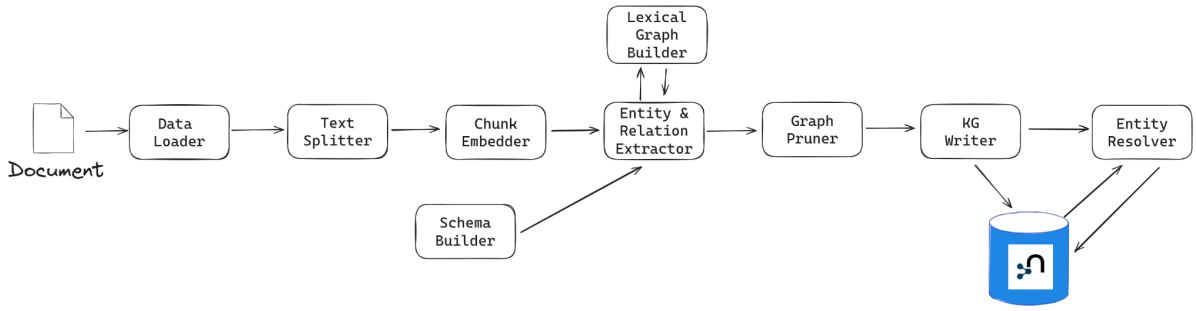
Hà Nội – 2026

Báo Cáo Tuần 5

Constructing Knowledge Graphs with Neo4j GraphRAG for Python

The SimpleKGPipeline class provides a pipeline which implements a series of steps to create a knowledge graph from unstructured data:

1. Load the text
2. Split the text into chunks
3. Create embeddings for each chunk
4. Extract entities from the chunks
5. Write the data to a Neo4j database



Create the knowledge graph

SimpleKGPipeline

```
import os
from dotenv import load_dotenv
load_dotenv()

import asyncio

from neo4j import GraphDatabase
from neo4j_graphrag.llm import OpenAILLM
from neo4j_graphrag.embeddings import OpenAIEmbeddings
from neo4j_graphrag.experimental.pipeline.kg_builder import SimpleKGPipeline

neo4j_driver = GraphDatabase.driver(
    os.getenv("NEO4J_URI"),
    auth=(os.getenv("NEO4J_USERNAME"), os.getenv("NEO4J_PASSWORD"))
)
```

```
neo4j_driver.verify_connectivity()

llm = OpenAILLM(
    model_name="gpt-5-nano",
    model_params={
        "reasoning_effort": "minimal"
    }
)

embedder = OpenAIEmbeddings(
    model="text-embedding-ada-002"
)

kg_builder = SimpleKGPipeline(
    llm=llm,
    driver=neo4j_driver,
    neo4j_database=os.getenv("NEO4J_DATABASE"),
    embedder=embedder,
    from_pdf=True,
)

pdf_file =
"./genai-graphrag-python/data/genai-fundamentals_1-generative-ai_1-what-is-genai.p
df"
result = asyncio.run(kg_builder.run_async(file_path=pdf_file))
print(result.result)
```

The code loads a single pdf file, data/genai-fundamentals_1-generative-ai_1-what-is-genai.pdf, and run the pipeline to create a knowledge graph in Neo4j.

The PDF document contains the content from the What is Generative AI? lesson.

Breaking down the code, you can see the following steps:

1. Create a connection to Neo4j:

```
neo4j_driver = GraphDatabase.driver(
    os.getenv("NEO4J_URI"),
    auth=(os.getenv("NEO4J_USERNAME"), os.getenv("NEO4J_PASSWORD"))
)
neo4j_driver.verify_connectivity()
```

2. Instantiate an LLM model:

```
llm = OpenAILLM(
    model_name="gpt-5-nano",
    model_params={
        "reasoning_effort": "minimal"
    }
)
```

3. Create an embedding model:

```
embedder = OpenAIEmbeddings(
    model="text-embedding-ada-002"
)
```

4. Setup the SimpleKGPipeline:

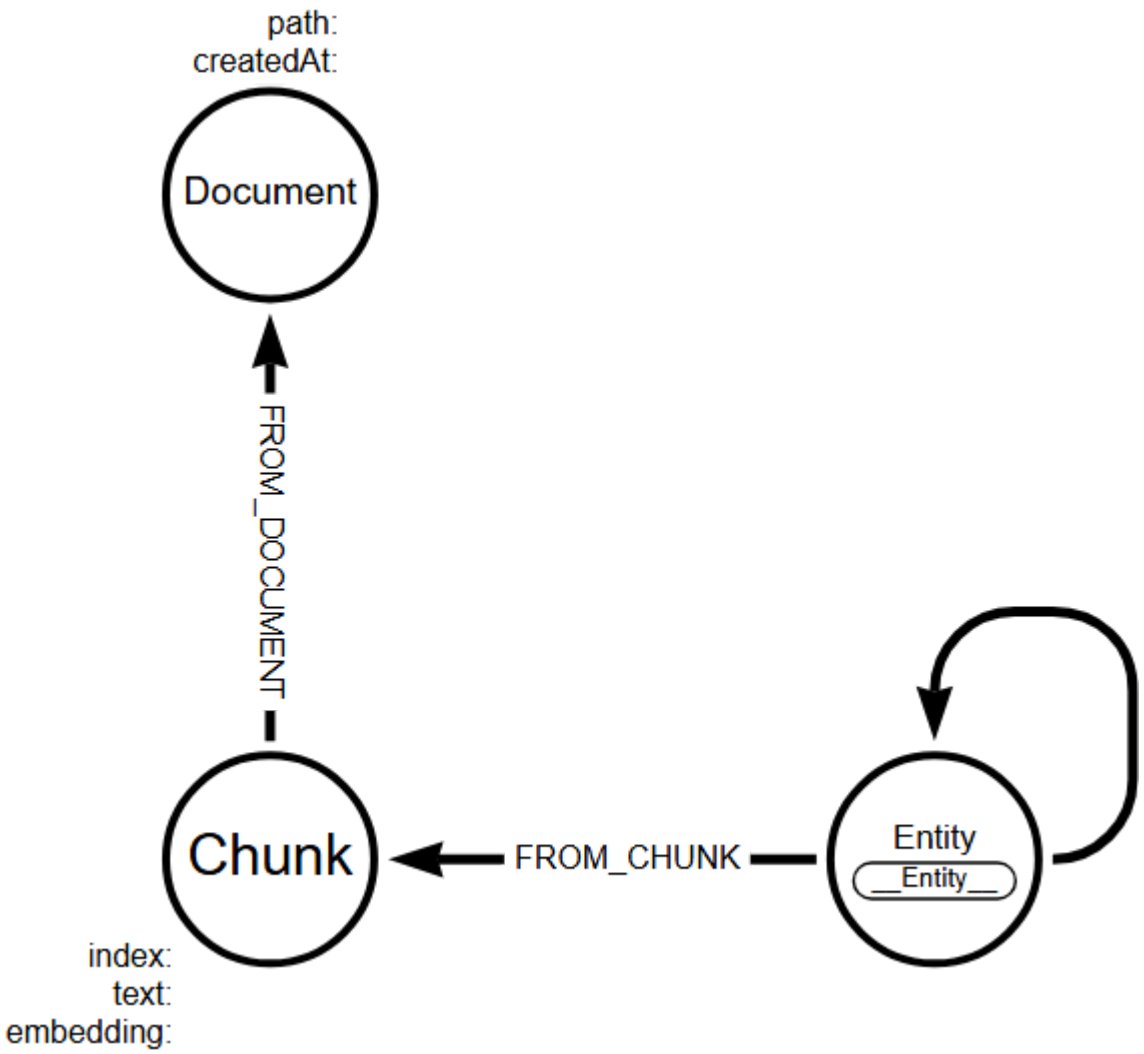
```
kg_builder = SimpleKGPipeline(
    llm=llm,
    driver=neo4j_driver,
    neo4j_database=os.getenv("NEO4J_DATABASE"),
    embedder=embedder,
    from_pdf=True,
)
```

5. Run the pipeline to create the graph from a single PDF file:

```
pdf_file =
"./genai-graphrag-python/data/genai-fundamentals_1-generative-ai_1-what-is-genai.p
df"
result = asyncio.run(kg_builder.run_async(file_path=pdf_file))
print(result.result)
```

Explore the Knowledge Graph

The SimpleKGPipeline creates the following default graph model:



The Entity nodes represent the entities extracted from the text chunks. Relevant properties are extracted from the chunk and associated with the entity nodes.

Chunk size

The graph created by the SimpleKGPipeline is based on chunks of text extracted from the documents.

By default, the chunk size is quite large, which may result in fewer, larger chunks.

The larger the chunk size, the more context the LLM has when extracting entities and relationships, but it may also lead to less granular data.

In this lesson, you will modify the SimpleKGPipeline to use a different chunk size.

Text Splitter Chunk Size

To modify the chunk size you will need to create a FixedSizeSplitter object and pass it to the SimpleKGPipeline when creating the pipeline instance:

1. Modify the genai-graphrag-python/kg_builder.py file to import the FixedSizeSplitter class and create an instance with a chunk size of 500 characters:

```
from neo4j_graphrag.experimental.components.text_splitters.fixed_size_splitter
import FixedSizeSplitter

text_splitter = FixedSizeSplitter(chunk_size=500, chunk_overlap=100)
```

2. Update the SimpleKGPipeline instantiation to use the custom text splitter:

```
kg_builder = SimpleKGPipeline(
    llm=llm,
    driver=neo4j_driver,
    neo4j_database=os.getenv("NEO4J_DATABASE"),
    embedder=embedder,
    from_pdf=True,
    text_splitter=text_splitter,
)
```

Define a schema

The knowledge graph you created is unconstrained, meaning that any entity or relationship can be created based on the data extracted from the text.

This can lead to graphs that are non-specific and may be difficult to analyze and query.

In this lesson, you will modify the SimpleKGPipeline to use a custom schema for the knowledge graph.

Schema

When you provide a schema to the SimpleKGPipeline, it will pass this information to the LLM instructing it to only identify those nodes and relationships.

This allows you to create a more structured and meaningful knowledge graph.

You define a schema by expressing the desired nodes, relationships, or patterns you want to extract from the text.

For example, you might want to extract the following information:

- nodes - Person, Organization, Location
- relationships - WORKS_AT, LOCATED_IN
- patterns - (Person)-[WORKS_AT]→(Organization), (Organization)-[LOCATED_IN]→(Location)

```
import os
from dotenv import load_dotenv
load_dotenv()

import asyncio

from neo4j import GraphDatabase
from neo4j_graphrag.llm import OpenAILLM
from neo4j_graphrag.embeddings import OpenAIEmbeddings
from neo4j_graphrag.experimental.pipeline.kg_builder import SimpleKGPipeline
from neo4j_graphrag.experimental.components.text_splitters.fixed_size_splitter
import FixedSizeSplitter

neo4j_driver = GraphDatabase.driver(
    os.getenv("NEO4J_URI"),
    auth=(os.getenv("NEO4J_USERNAME"), os.getenv("NEO4J_PASSWORD"))
)
neo4j_driver.verify_connectivity()

llm = OpenAILLM(
    model_name="gpt-5-nano",
    model_params={
        "reasoning_effort": "minimal"
```

```
    }
)

embedder = OpenAIEmbeddings(
    model="text-embedding-ada-002"
)

text_splitter = FixedSizeSplitter(chunk_size=500, chunk_overlap=100)


NODE_TYPES = [
    "Technology",
    "Concept",
    "Example",
    "Process",
    "Challenge",
    "Benefit",
    {
        "label": "Resource",
        "description": "A related learning resource such as a book, article, video, or
course.",
        "properties": [
            {"name": "name", "type": "STRING", "required": True},
            {"name": "url", "type": "STRING"}
        ]
    },
]

RELATIONSHIP_TYPES = [
    "RELATED_TO",
    "PART_OF",
    "USED_IN",
    "LEADS_TO",
    "HAS_CHALLENGE",
    "LEADS_TO",
    "CITES"
]

PATTERNS = [
```

```
    ("Technology", "RELATED_TO", "Technology"),
    ("Concept", "RELATED_TO", "Technology"),
    ("Example", "USED_IN", "Technology"),
    ("Process", "PART_OF", "Technology"),
    ("Technology", "HAS_CHALLENGE", "Challenge"),
    ("Concept", "HAS_CHALLENGE", "Challenge"),
    ("Technology", "LEADS_TO", "Benefit"),
    ("Process", "LEADS_TO", "Benefit"),
    ("Resource", "CITES", "Technology"),
]

kg_builder = SimpleKGPipeline(
    llm=llm,
    driver=neo4j_driver,
    neo4j_database=os.getenv("NEO4J_DATABASE"),
    embedder=embedder,
    from_pdf=True,
    text_splitter=text_splitter,
    schema={
        "node_types": NODE_TYPES,
        "relationship_types": RELATIONSHIP_TYPES,
        "patterns": PATTERNS
    },
)

pdf_file =
"./genai-graphrag-python/data/genai-fundamentals_1-generative-ai_1-what-is-genai.p
df"
result = asyncio.run(kg_builder.run_async(file_path=pdf_file))
print(result.result)
```

Add structured data to the knowledge graph

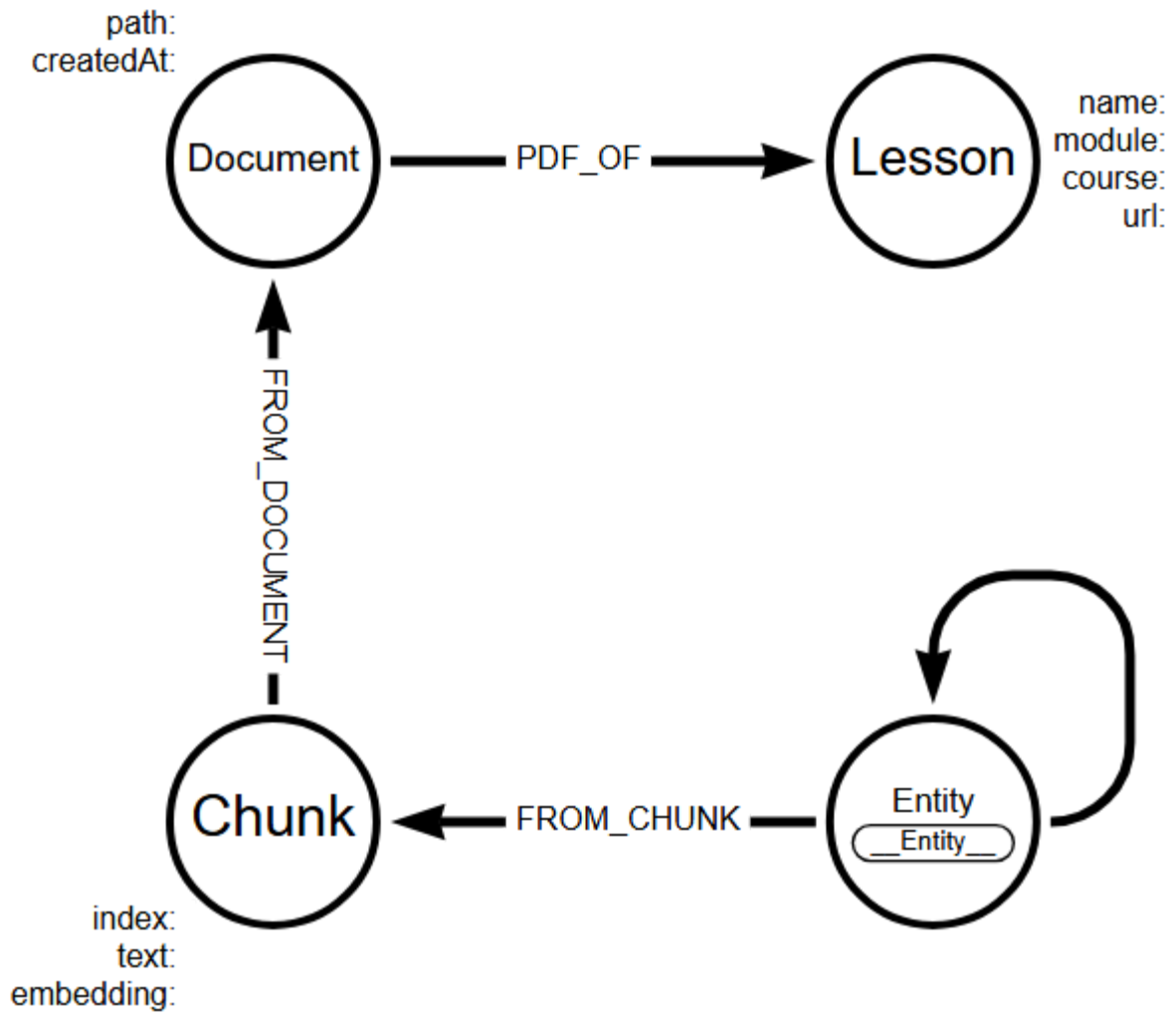
The knowledge graph you created is solely based on unstructured data extracted from documents. You may have access to structured data sources such as databases, CSV files, or APIs that contain valuable information relevant to your domain.

Combining the structured and unstructured data can enhance the knowledge graph's richness and usefulness.

Structured data source

The repository contains a sample CSV file genai-graphrag-python/data/docs.csv which contains metadata about the lessons the documents were created from.

You will create Lesson nodes from the structured data and connected them to the Document nodes using a PDF_OF relationship:



Load from CSV file

You can use the CSV file as input and a structured data source when creating the knowledge graph.

```
import os
from dotenv import load_dotenv
load_dotenv()

import asyncio
import csv

from neo4j import GraphDatabase
from neo4j_graphrag.llm import OpenAILLM
from neo4j_graphrag.embeddings import OpenAIEmbeddings
from neo4j_graphrag.experimental.pipeline.kg_builder import SimpleKGPipeline
from neo4j_graphrag.experimental.components.text_splitters.fixed_size_splitter
import FixedSizeSplitter

neo4j_driver = GraphDatabase.driver(
    os.getenv("NEO4J_URI"),
    auth=(os.getenv("NEO4J_USERNAME"), os.getenv("NEO4J_PASSWORD"))
)
neo4j_driver.verify_connectivity()

llm = OpenAILLM(
    model_name="gpt-5.2"
)

embedder = OpenAIEmbeddings(
    model="text-embedding-ada-002"
)

text_splitter = FixedSizeSplitter(chunk_size=500, chunk_overlap=100)

NODE_TYPES = [
    "Technology",
    "Concept",
    "Example",
    "Process",
    "Challenge",
    "Benefit",
    {
```

```
        "label": "Resource",
        "description": "A related learning resource such as a book, article, video, or
course.",
        "properties": [
            {"name": "name", "type": "STRING", "required": True},
            {"name": "link", "type": "STRING"}
        ]
    },
]

RELATIONSHIP_TYPES = [
    "RELATED_TO",
    "PART_OF",
    "USED_IN",
    "LEADS_TO",
    "HAS_CHALLENGE",
    "LEADS_TO",
    "CITES"
]

PATTERNS = [
    ("Technology", "RELATED_TO", "Technology"),
    ("Concept", "RELATED_TO", "Technology"),
    ("Example", "USED_IN", "Technology"),
    ("Process", "PART_OF", "Technology"),
    ("Technology", "HAS_CHALLENGE", "Challenge"),
    ("Concept", "HAS_CHALLENGE", "Challenge"),
    ("Technology", "LEADS_TO", "Benefit"),
    ("Process", "LEADS_TO", "Benefit"),
    ("Resource", "CITES", "Technology"),
]

kg_builder = SimpleKGPipeline(
    llm=llm,
    driver=neo4j_driver,
    neo4j_database=os.getenv("NEO4J_DATABASE"),
    embedder=embedder,
    from_pdf=True,
    text_splitter=text_splitter,
```

```
    schema={
        "node_types": NODE_TYPES,
        "relationship_types": RELATIONSHIP_TYPES,
        "patterns": PATTERNS
    },
)

data_path = "./genai-graphrag-python/data/"

docs_csv = csv.DictReader(
    open(os.path.join(data_path, "docs.csv"), encoding="utf8", newline='')
)

cypher = """
MATCH (d:Document {path: $pdf_path})
MERGE (l:Lesson {url: $url})
SET l.name = $lesson,
    l.module = $module,
    l.course = $course
MERGE (d)-[:PDF_OF]->(l)
"""

for doc in docs_csv:

    # Create the complete PDF path
    doc["pdf_path"] = os.path.join(data_path, doc["filename"])
    print(f"Processing document: {doc['pdf_path']}")

    # Entity extraction and KG population
    result = asyncio.run(
        kg_builder.run_async(
            file_path=os.path.join(doc["pdf_path"])
        )
    )

    # Create structured graph
    records, summary, keys = neo4j_driver.execute_query(
        cypher,
        parameters_=doc,
```

```
        database_=os.getenv("NEO4J_DATABASE")
    )
    print(result, summary.counters)
```

The key differences are:

1. The docs.csv file is loaded using csv.DictReader to read each row as a dictionary:

```
data_path = "./genai-graphrag-python/data/"

docs_csv = csv.DictReader(
    open(os.path.join(data_path, "docs.csv"), encoding="utf8", newline='')
)
```

2. The path of the PDF document is constructed using the filename field from the CSV:

```
    # Create the complete PDF path
    doc["pdf_path"] = os.path.join(data_path, doc["filename"])
    print(f"Processing document: {doc['pdf_path']}")
```

3. A cypher statement is defined to create Lesson nodes with properties from the CSV data:

```
cypher = """
MATCH (d:Document {path: $pdf_path})
MERGE (l:Lesson {url: $url})
SET l.name = $lesson,
    l.module = $module,
    l.course = $course
MERGE (d)-[:PDF_OF]->(l)
"""
```

The pdf_path is used as the key to match the Document nodes created from the PDF files.

4. A Lesson node is created for each document using the cypher statement and the CSV data:

```
# Create structured graph
records, summary, keys = neo4j_driver.execute_query(
    cypher,
    parameters_=doc,
    database_=os.getenv("NEO4J_DATABASE")
)
```

Vector + Cypher retriever

The chunks in the knowledge graph include vector embeddings that allow for similarity search based on vector distance.

In this lesson you will create a vector retriever that uses these embeddings to find the most relevant chunks for a given query.

The retriever can then use the structured and unstructured data in the knowledge graph to provide additional context.

Create the Vector Index

You will need to create a vector index on the Chunk nodes embedding properties:

```
CREATE VECTOR INDEX chunkEmbedding IF NOT EXISTS
FOR (n:Chunk)
ON n.embedding
OPTIONS {indexConfig: {
 `vector.dimensions`: 1536,
 `vector.similarity_function`: 'cosine'
}};
```

A vector index named chunkEmbedding will be created for nodes with a Chunk label, indexing the embedding property. The index is configured to 1536 dimensions (as provided by the text-embedding-ada-002 embedding model) and use cosine similarity for distance calculations.

Create a Vector + Cypher GraphRAG pipeline

The neo4j_graphrag package includes a VectorCypherRetriever class that combines vector similarity search with Cypher retrieval.

You can use this retriever to create a GraphRAG pipeline to:

1. Perform a vector similarity search to find the most relevant chunks for a given query.
2. Use a Cypher query to add additional information to the context.
3. Pass the context to an LLM to generate a response to the original query.

```
import os
from dotenv import load_dotenv
load_dotenv()

from neo4j import GraphDatabase
from neo4j_graphrag.embeddings.openai import OpenAIEmbeddings
from neo4j_graphrag.retrievers import VectorCypherRetriever
from neo4j_graphrag.llm import OpenAILLM
from neo4j_graphrag.generation import GraphRAG

# Connect to Neo4j database
driver = GraphDatabase.driver(
    os.getenv("NEO4J_URI"),
    auth=(
        os.getenv("NEO4J_USERNAME"),
        os.getenv("NEO4J_PASSWORD")
    )
)

# Create embedder
embedder = OpenAIEmbeddings(model="text-embedding-ada-002")

# Define retrieval query
retrieval_query = """
```

```
RETURN node.text as text, score
"""

# Create retriever
retriever = VectorCypherRetriever(
    driver,
    neo4j_database=os.getenv("NEO4J_DATABASE"),
    index_name="chunkEmbedding",
    embedder=embedder,
    retrieval_query=retrieval_query,
)

#  Create the LLM
llm = OpenAILLM(
    model_name="gpt-5.2"
)

# Create GraphRAG pipeline
rag = GraphRAG(retriever=retriever, llm=llm)

# Search
query_text = "What is a knowledge graphs and where can I learn about them?"

response = rag.search(
    query_text=query_text,
    retriever_config={"top_k": 5},
    return_context=True
)

print(response.answer)

# Close the database connection
driver.close()
```

The retriever is configured to use the chunkEmbedding vector index you just created.

```
retriever = VectorCypherRetriever(
    driver,
    neo4j_database=os.getenv("NEO4J_DATABASE"),
    index_name="chunkEmbedding",
```

```
    embedder=embedder,
    retrieval_query=retrieval_query,
)
```

When you run the code:

1. The VectorCypherRetriever uses the vector index to find chunks similar to the query:
   "What is a knowledge graphs and where can I learn about them?"
2. The GraphRAG pipeline passes the text from those chunks as context to the LLM.
3. The response from the LLM is printed:
   A knowledge graph is an organized representation of real-world entities…

Retrieval Cypher Query

The VectorCypherRetriever also allows you to define a Cypher query to retrieve additional context from the knowledge graph.

Adding additional context can help the LLM generate more accurate responses.

Update the retrieval_query to add additional information about the lessons, technologies, and concepts related to the chunks:

```
retrieval_query = """
MATCH (node)-[:FROM_DOCUMENT]->(d)-[:PDF_OF]->(lesson)
RETURN DISTINCT
    node.text as text, score,
    lesson.url as lesson_url,
    collect { MATCH (node)<-[:FROM_CHUNK]-(e:Technology) RETURN e.name }
as technologies,
    collect { MATCH (node)<-[:FROM_CHUNK]-(e:Concept) RETURN e.name } as
concepts
"""
```

The retriever will execute the Cypher query adding more context.

Running the code again for the same query, *"What is a knowledge graphs and where can I learn about them?"*, will produce a more detailed response:

_A knowledge graph is an organized representation of real-world entities… Knowledge graphs are also useful for Generative AI because they provide structured, interconnected data.

Generalize entity retrieval

The retriever currently uses the knowledge graph to add additional context related to technologies and concepts. The specific entities allow for targeted retrieval, however you may also want to generalize the retrieval to include all related entities.

You can use the node labels and relationship types to create a response that includes details about the entities.

This cypher query retrieves all related entities between the chunks:

```
MATCH (c:Chunk)<-[:FROM_CHUNK]-(entity)-[r]->(other)-[:FROM_CHUNK]->()
RETURN DISTINCT
  labels(entity)[2], entity.name, entity.type, entity.description,
  type(r),
  labels(other)[2], other.name, other.type, other.description
```

The output uses the node labels, properties, and relationship types to output rows which form statements such as:

- Concept "Semantic Search" RELATED_TO Technology "Vector Indexes"
- Technology "Retrieval Augmented Generation" HAS_CHALLENGE "Understanding what the user is asking for and finding the correct information to pass to the LLM"`

These statements can be used to create additional context for the LLM to generate responses.

Modify the retrieval_query to include all entities associated with the chunk:

```
retrieval_query = """
MATCH (node)-[:FROM_DOCUMENT]->(d)-[:PDF_OF]->(lesson)
RETURN
   node.text as text, score,
   lesson.url as lesson_url,
   collect {
      MATCH
(node)<-[:FROM_CHUNK]-(entity)-[r]->(other)-[:FROM_CHUNK]->()
      WITH toStringList([
         labels(entity)[2],
         entity.name,
         entity.type,
         entity.description,
         type(r),
         labels(other)[2],
         other.name,
         other.type,
         other.description
         ]) as values
      RETURN reduce(acc = "", item in values | acc || coalesce(item || ' ', ''))
   } as associated_entities
"""
```

Text to Cypher retriever

The Text2CypherRetriever retriever allows you to create GraphRAG pipelines that can answer natural language questions by generating and executing Cypher queries against the knowledge graph.

Using text to cypher retrieval can help you get precise information from the knowledge graph based on user questions. For example, how many lessons are in a course, what concepts are covered in a module, or how technologies relate to each other.

In this lesson, you will create a text to cypher retriever and use it to answer questions about the data in knowledge graph.

Create a Text2CypherRetriever GraphRAG pipeline

```
import os
from dotenv import load_dotenv
load_dotenv()

from neo4j import GraphDatabase
from neo4j_graphrag.llm import OpenAILLM
from neo4j_graphrag.generation import GraphRAG
from neo4j_graphrag.retrievers import Text2CypherRetriever

# Connect to Neo4j database
driver = GraphDatabase.driver(
    os.getenv("NEO4J_URI"),
    auth=(
        os.getenv("NEO4J_USERNAME"),
        os.getenv("NEO4J_PASSWORD")
    )
)

llm = OpenAILLM(
    model_name="gpt-5.2"
)

# Cypher examples as input/query pairs
examples = [
    "USER INPUT: 'Find a node with the name $name?' QUERY: MATCH (node)
WHERE toLower(node.name) CONTAINS toLower($name) RETURN node.name
AS name, labels(node) AS labels",
]

# Build the retriever
retriever = Text2CypherRetriever(
    driver=driver,
    neo4j_database=os.getenv("NEO4J_DATABASE"),
    llm=llm,
    examples=examples,
)
```

```
rag = GraphRAG(
    retriever=retriever,
    llm=llm
)

query_text = "How many technologies are mentioned in the knowledge graph?"

response = rag.search(
    query_text=query_text,
    return_context=True
    )

print(response.answer)
print("CYPHER :", response.retriever_result.metadata["cypher"])
print("CONTEXT:", response.retriever_result.items)

driver.close()
```

The retriever is configured to use your database connection and given an example of how to query nodes by name:

```
# Cypher examples as input/query pairs
examples = [
    "USER INPUT: 'Find a node with the name $name?' QUERY: MATCH (node)
WHERE toLower(node.name) CONTAINS toLower($name) RETURN node.name
AS name, labels(node) AS labels",
]

# Build the retriever
retriever = Text2CypherRetriever(
    driver=driver,
    neo4j_database=os.getenv("NEO4J_DATABASE"),
    llm=llm,
    examples=examples,
)
```

The response includes the Cypher statement that the LLM generated and the results from executing the query:

```
print(response.answer)
print("CYPHER :", response.retriever_result.metadata["cypher"])
print("CONTEXT:", response.retriever_result.items)
```